

BACKEND DEVELOPER INTERN ASSESSMENT

Factory Traffic Management System - IoT / Factory Automation Technical Assignment

Expected Time	4-5 hours	Primary Focus	Backend engineering
Frontend	Functional dashboard	Submission	Source + README
Database	Candidate choice	IoT	REST simulation; MQTT optional

Important: The specification intentionally contains incomplete, ambiguous, or conflicting requirements. Candidates are expected to identify issues, make reasonable assumptions, and document their decisions. Completing every feature is not required; prioritization is part of the assessment.

1. Objective

Design and implement a small factory traffic-management application for internal roads inside a garment manufacturing facility.

The system will receive vehicle-detection events, maintain traffic queues, control traffic signals, allow manual intervention, handle emergencies and device failures, keep event history, survive application restarts, and expose a simple frontend dashboard.

The system should be treated as a small event-driven control system, not simply a CRUD application.

The assignment is intended to evaluate engineering judgment in addition to coding ability. Candidates should identify unclear or unsafe requirements rather than blindly implementing them.

The solution should demonstrate reasonable separation between:

- API / communication layer
- Traffic-control domain logic
- Persistence
- Physical-controller communication
- Frontend

The core traffic-control logic should not depend directly on HTTP, MQTT, frontend code, or a specific database technology.

2. Factory Scenario

The factory has internal roads used by multiple types of vehicles:

- Forklifts
- Delivery trucks
- Material-carrying vehicles
- Employee transport vehicles
- Emergency vehicles

For the assessment, the core implementation may focus on Junction A. Junction A has traffic signals for NORTH, SOUTH, EAST, and WEST.

The architecture should allow additional junctions such as Junction B, C, and D to be introduced later without rewriting the traffic-control engine.

For this simplified assessment, assume two basic traffic phases:

NORTH + SOUTH

EAST + WEST

Conflicting phases must never receive GREEN simultaneously. Junction-specific configuration should preferably not be deeply hard-coded into business logic.

3. Traffic Signal States

Supported physical signal states:

- RED
- YELLOW
- GREEN

Normal GREEN duration should be approximately 30 seconds. YELLOW should remain active for approximately 5 seconds before transitioning to RED.

The implementation may introduce an internal state such as ALL_RED for safe clearance between conflicting phases.

Example:

```
North = GREEN
South = GREEN
East  = RED
West  = RED
```

A safe transition from NORTH/SOUTH to EAST/WEST should conceptually behave like:

```
NORTH/SOUTH GREEN
|
NORTH/SOUTH YELLOW
|
ALL RED
|
EAST/WEST GREEN
```

The following safety rules must always hold:

1. Conflicting traffic phases must never simultaneously receive GREEN.
2. A GREEN phase must never transition directly into a conflicting GREEN phase.
3. Manual or emergency priority must not bypass the safe signal-transition sequence.
4. Invalid or unexpected commands must not place the junction into an unsafe state.

The backend should control these rules centrally. Clients should request an operation or intent; they should not be able to bypass domain logic by directly writing arbitrary signal states.

4. Vehicle Detection

Sensors near the junction send events to the backend when vehicles arrive.

Example:

```
{
  "event_id": "evt-10001",
  "junction_id": "A",
  "direction": "NORTH",
  "event_type": "VEHICLE_ARRIVED",
  "vehicle_id": "VH-501",
  "vehicle_type": "FORKLIFT",
  "sequence_no": 1501,
  "timestamp": "2026-10-05T10:15:20Z"
}
```

Supported vehicle types:

- FORKLIFT
- TRUCK
- EMPLOYEE_VEHICLE
- EMERGENCY

Vehicle priority should normally be:

```
EMERGENCY > TRUCK > FORKLIFT > EMPLOYEE_VEHICLE
```

Higher-priority vehicles should normally receive faster clearance. The system should also support a vehicle leaving or being cleared from a queue.

Example:

```
{
  "event_id": "evt-10002",
  "junction_id": "A",
  "direction": "NORTH",
  "event_type": "VEHICLE_CLEARED",
  "vehicle_id": "VH-501",
  "sequence_no": 1502,
  "timestamp": "2026-10-05T10:16:10Z"
}
```

The backend should reasonably handle:

- Duplicate events
- Delayed events
- Out-of-order events
- Repeated submissions
- Invalid events
- Unknown junctions
- Unknown vehicle types
- Missing or malformed fields

Processing the same event_id more than once must not incorrectly change the queue multiple times. The candidate should explain how sensor timestamps, server-received timestamps, and sequence numbers are interpreted.

5. Queue Management

The backend should maintain the waiting traffic for each direction.

Example:

```
{
  "junction_id": "A",
  "queues": {
    "NORTH": 5,
    "SOUTH": 2,
    "EAST": 8,
    "WEST": 0
  }
}
```

Queue state should be based on valid vehicle events rather than arbitrary frontend values. The system should prevent conditions such as queue = -1.

A direction containing more vehicles should normally receive higher scheduling priority. Vehicle type should also influence scheduling. The system should prevent lower-priority traffic from waiting indefinitely.

Candidates may choose their own scheduling/scoring strategy but should document it. Example factors may include:

```
Queue Size + Vehicle Priority + Waiting Time + Starvation Protection
```

The exact weighting is intentionally unspecified.

6. Automatic Traffic Control

In automatic mode, the application should:

- Evaluate current queues
- Evaluate vehicle priorities
- Consider waiting duration
- Select the next safe traffic phase
- Execute safe signal transitions

- Avoid unnecessary switching
- Prevent starvation
- Continue processing incoming events
- Remain consistent when multiple events arrive close together

The exact scheduling algorithm is not specified. Explain the chosen strategy in the README.

Traffic-control state should preferably be represented using an explicit state machine or equivalent deterministic mechanism. Possible junction operating modes include:

```
AUTOMATIC
MANUAL
EMERGENCY
DEGRADED / FAILURE
```

Traffic decisions affecting the same junction must not produce conflicting results because two requests were processed concurrently.

The candidate should choose and explain an appropriate consistency strategy. Possible approaches may include:

- Database transaction/locking
- Optimistic concurrency control
- Serialized per-junction processing
- Actor/event-loop model
- Other reasonable approaches

The implementation should not rely on long blocking sleep() operations inside request handlers to manage signal timing.

7. Emergency Vehicle Handling

Emergency vehicles receive the highest priority. If an emergency vehicle approaches from EAST while NORTH/SOUTH currently has GREEN, the system should immediately begin safe emergency preemption.

This does not mean EAST becomes GREEN immediately.

Example:

```
NORTH/SOUTH GREEN
|
Emergency EAST detected
|
NORTH/SOUTH YELLOW
|
ALL RED
|
EAST/WEST GREEN
```

Emergency priority must never violate traffic-safety invariants.

The implementation should consider:

- Emergency during normal traffic
- Emergency during a manual override
- Multiple emergency vehicles
- Emergency vehicles from conflicting directions
- Repeated emergency events
- Emergency completion
- Emergency timeout or stale emergency events

The exact policy for competing emergency requests is intentionally not fully specified. Make and document a reasonable decision.

8. Manual Control

Factory administrators should be able to request manual traffic control. Manual control should override normal automatic scheduling but should still use the same safety-transition logic.

Instead of allowing a client to directly force an arbitrary signal state, a command-oriented API is preferred.

```
POST /api/junctions/A/commands

{
  "command": "MANUAL_GREEN_REQUEST",
  "direction": "WEST"
}
```

The backend should determine the safe transition necessary to execute the request.

Example:

```
NORTH/SOUTH GREEN
|
NORTH/SOUTH YELLOW
|
ALL RED
|
EAST/WEST GREEN
```

Provide a way to return the junction to automatic control.

```
{
  "command": "RETURN_TO_AUTOMATIC"
}
```

The requirements do not define:

- How long manual control remains active
- What happens when two administrators issue commands simultaneously
- Whether emergency mode overrides manual mode

- Whether manual commands expire
- What happens when an administrator disconnects

Make and document reasonable decisions.

9. Signal, Sensor and Controller Failure

Traffic signals, sensors, or physical junction controllers may fail.

Example status event:

```
{
  "event_id": "status-301",
  "junction_id": "A",
  "device_type": "SIGNAL_CONTROLLER",
  "direction": "SOUTH",
  "status": "OFFLINE",
  "timestamp": "2026-10-05T10:30:00Z"
}
```

The system should distinguish between the state the backend wants and the state confirmed by the physical controller.

```
Desired State = GREEN
Actual State = RED
```

Sending a command does not automatically mean the physical device successfully executed it.

Example backend command:

```
{
  "command_id": "cmd-8001",
  "junction_id": "A",
  "direction": "EAST",
  "requested_state": "GREEN"
}
```

Example controller acknowledgement:

```
{
  "command_id": "cmd-8001",
  "status": "ACK",
  "actual_state": "GREEN",
  "timestamp": "2026-10-05T10:31:00Z"
}
```

The implementation should reasonably consider:

- ACK received
- ACK delayed
- ACK never received
- Controller offline
- Sensor offline
- Unknown physical state
- Command failure
- Controller reconnecting
- Backend restarting while a command is pending

A controller that does not confirm a requested state must not automatically be assumed to have executed it. Failure handling should prioritize safety. Clearly document assumptions and fallback behavior.

10. Minimum Backend APIs

10.1 Junctions

```
GET /api/junctions
GET /api/junctions/id
POST /api/junctions
```

10.2 Sensor Events

```
POST /api/sensor-events
```

Example:

```
{
  "event_id": "evt-10001",
  "junction_id": "A",
  "direction": "NORTH",
  "event_type": "VEHICLE_ARRIVED",
  "vehicle_id": "VH-501",
  "vehicle_type": "TRUCK",
  "sequence_no": 1501,
  "timestamp": "2026-10-05T10:15:20Z"
}
```

Duplicate submission of the same logical event should not modify traffic state multiple times.

10.3 Junction Status

```
GET /api/junctions/id/status
```

Example:

```
{
  "junction_id": "A",
  "mode": "AUTOMATIC",
  "phase": "NORTH_SOUTH",
  "controller_status": "ONLINE",
  "desired_signals": {"NORTH": "GREEN", "SOUTH": "GREEN", "EAST": "RED", "WEST": "RED"},
  "actual_signals": {"NORTH": "GREEN", "SOUTH": "GREEN", "EAST": "RED", "WEST": "RED"},
  "queues": {"NORTH": 3, "SOUTH": 1, "EAST": 7, "WEST": 2}
}
```

10.4 Manual / Control Commands

POST /api/junctions/:id/commands

Example:

```
{
  "command": "MANUAL_GREEN_REQUEST",
  "direction": "WEST"
}
```

or:

```
{
  "command": "RETURN_TO_AUTOMATIC"
}
```

10.5 Controller Events / Acknowledgements

A reasonable API should exist for simulating physical-controller responses.

POST /api/controller-events

Example:

```
{
  "command_id": "cmd-8001",
  "junction_id": "A",
  "status": "ACK",
  "actual_state": "GREEN"
}
```

10.6 History

GET /api/junctions/:id/history

You may add or change endpoints if you believe another API design is more appropriate. Explain significant changes.

11. Traffic History / Audit Log

Keep a history of important system events, including:

- Vehicle detected
- Vehicle cleared
- Duplicate/rejected sensor event
- Signal transition started
- Signal state requested
- Signal state confirmed
- Emergency detected
- Emergency cleared
- Manual override
- Return to automatic mode
- Controller acknowledgement
- Controller timeout
- Device/signal/sensor failure
- Mode change

Example:

```
{
  "junction_id": "A",
  "event_type": "SIGNAL_CHANGED",
  "direction": "NORTH",
  "previous_state": "RED",
  "new_state": "GREEN",
  "command_id": "cmd-8001",
  "timestamp": "2026-10-05T10:20:00Z"
}
```

Audit history should make it reasonably possible to understand why the system reached its current state.

12. Database and Persistence

Use any reasonable database, for example PostgreSQL, MySQL, MongoDB, or SQLite. Store the configuration and traffic information you believe is necessary.

Traffic information and important history should remain available after a server restart. Candidates should decide which data must be persisted and which data may safely be reconstructed.

The design should consider persistence of information such as:

- Junction configuration
- Queue state
- Processed event IDs

- Current operating mode
- Current/desired signal state
- Controller-confirmed state
- Emergency state
- Manual override state
- Pending commands
- Important timestamps
- Audit history

A restart must not cause the application to blindly assume that previously requested physical signal states are still correct.

The candidate should explain how recovery works if the application restarts during a transition such as:

```
GREEN -> YELLOW -> ALL RED -> GREEN
```

or while waiting for a physical-controller acknowledgement.

13. IoT Communication

Physical traffic controllers will eventually communicate with the backend using MQTT. For this assessment, you may either implement MQTT or simulate controllers using REST APIs.

The application should be structured so MQTT can later replace or supplement the REST simulator without rewriting the core traffic-control logic.

A reasonable architecture may resemble:

```
REST / MQTT
|
Application Layer
|
Traffic Domain Engine
|
Controller Interface / Port
|
REST Simulator / MQTT Adapter
```

The domain engine should not contain MQTT-specific or HTTP-specific traffic-control rules. Commands sent to physical controllers should preferably have unique identifiers so that acknowledgements can be correlated.

```
command_id = cmd-8001
```

The candidate should decide how command timeout, retry, duplicate acknowledgement, and failure are handled.

14. Frontend Dashboard

Create a simple web dashboard that consumes the backend APIs. Frontend functionality is required for demonstration, but visual polish is secondary to backend correctness.

You may use React, Next.js, Vue, Angular, Plain HTML/CSS/JavaScript, or another reasonable frontend framework.

14.1 Dashboard Overview

Show the current junction state. At minimum display:

- Signal states
- Queue size by direction
- Junction/controller status
- Current traffic-control mode
- Current phase
- Emergency indication
- Failure indication

14.2 Junction Detail View

The user should be able to open Junction A and view:

- Desired signal state
- Actual/confirmed signal state
- Current queues
- Current mode
- Active alerts
- Pending controller command, if applicable

Suggested modes include:

```
AUTOMATIC
MANUAL
EMERGENCY
FAILURE / DEGRADED
```

14.3 Visual Intersection

Provide a simple visual representation of the intersection. Complex graphics are not required.

```

      NORTH
      GREEN
      |
WEST RED -----+----- RED EAST
      |
      GREEN
      SOUTH
```

The visual state must come from backend state. The frontend must not independently decide traffic sequencing.

14.4 Manual Traffic Control

Provide controls allowing an administrator to request manual traffic control. Clearly indicate when a manual override is active. Provide a way to return the junction to automatic mode.

The frontend should request commands from the backend rather than directly manipulating independent frontend signal state.

14.5 Emergency Display

When an emergency vehicle is detected, clearly show the junction, direction, emergency mode, and current transition/state. The UI should display backend decisions rather than independently deciding emergency behavior.

14.6 Failure Display

Display clear warnings for:

- Controller offline
- Signal failure
- Sensor failure
- Desired/actual state mismatch
- Command timeout
- Unknown device state

Possible statuses include:

```

ONLINE
OFFLINE
DEGRADED
WARNING
UNKNOWN

```

14.7 Recent Activity

Show a recent history/activity list using backend history data. Filtering is optional.

14.8 Traffic Simulation

Provide a simple form that allows the evaluator to simulate:

- Vehicle arrival
- Vehicle clearance
- Vehicle type
- Direction
- Emergency vehicle
- Controller status
- Controller acknowledgement

The evaluator should be able to demonstrate the main system behavior without physical hardware.

14.9 Dashboard Refresh

Traffic information should update without requiring a manual browser reload. You may use Polling, Server-Sent Events, WebSocket, or another reasonable approach. Real-time technology is not mandatory. Explain your choice.

14.10 Frontend Error Handling

The UI should handle common errors without crashing, including:

- Backend unavailable
- API request failure
- Invalid junction
- Invalid command
- Failed manual override
- Failed sensor submission
- Controller offline
- Missing/incomplete data

15. Functional Scenarios to Demonstrate

- 1. Normal Traffic:** Several vehicles arrive from different directions and the system determines which phase to serve.
- 2. Priority Traffic:** A TRUCK or FORKLIFT affects scheduling compared with ordinary employee traffic.
- 3. Emergency Preemption:** An emergency vehicle arrives while a conflicting phase is GREEN. The system must perform a safe transition.
- 4. Manual Override:** An administrator takes control of Junction A and later returns it to automatic mode.
- 5. Duplicate Event:** The exact same sensor event is submitted twice. Queue state must remain correct.
- 6. Vehicle Clearance:** A vehicle arrives and later sends a VEHICLE_CLEARED event. Queue state changes correctly.
- 7. Controller Failure:** The backend requests a signal change but the controller does not acknowledge it or reports OFFLINE. The application responds safely.
- 8. Restart:** The backend restarts while persisted data exists. Important state/history remains available according to the candidate's design.
- 9. Concurrent Events:** Events affecting the same junction occur almost simultaneously.

Example concurrent sequence:

```

T = 0 ms  NORTH truck arrives
T = 4 ms  EAST emergency arrives
T = 8 ms  Administrator requests WEST manual control
T = 12 ms Duplicate EAST emergency event arrives
T = 17 ms Controller ACK arrives

```

The system must remain internally consistent and must never create conflicting GREEN states.

16. Non-Functional Expectations

The implementation should demonstrate:

- Clear separation between API/controller code and traffic-domain logic
- Reasonable input validation
- Appropriate HTTP status codes
- Idempotent processing of duplicate sensor events
- Concurrency/race-condition awareness
- Safe traffic-state transitions
- Persistence/recovery awareness
- Useful logging
- Error handling
- Readable code structure
- Reasonable naming
- Testable domain logic
- Separation of desired and controller-confirmed physical state

Core traffic logic should preferably be testable without requiring:

- HTTP requests
- A browser
- MQTT
- A physical controller

Avoid unnecessarily coupling traffic decisions to:

HTTP handlers
Database queries
Frontend state
MQTT callbacks

No production-grade authentication, deployment environment, or pixel-perfect frontend styling is required unless you choose to add them.

17. Intentionally Incomplete / Ambiguous Areas

Not every rule is fully specified. This is intentional. Examples of decisions you may need to make include:

- What exactly counts as a conflicting traffic movement?
- How is maximum waiting time calculated?
- What happens if a VEHICLE_CLEARED event arrives without a corresponding arrival?
- How should delayed sensor events be treated?
- How should out-of-order events be treated?
- How should duplicate events be detected?
- Is event_id, sequence_no, or another mechanism authoritative?
- Which timestamp is authoritative: sensor time or server time?
- How long does manual override remain active?
- What happens when two administrators issue commands simultaneously?
- Should emergency mode override manual mode?
- What happens if two emergency vehicles arrive from conflicting directions?
- When is an emergency considered cleared?
- How long should the backend wait for a controller acknowledgement?
- Should controller commands be retried?
- How should duplicate ACK messages be treated?
- What happens when the controller reconnects?
- What happens if desired state and actual state disagree?
- What should happen when the backend loses communication with the controller?
- How should signal timers behave after a server restart?
- How should the application recover if it restarts in the middle of a signal transition?

You are not expected to ask for clarification during the assessment. Make sensible assumptions, implement what you can, and document your reasoning.

18. Expected Submission

Submit:

1. Backend source code
2. Frontend source code
3. README with setup/run instructions
4. Database schema or migration files
5. API documentation or Postman collection
6. Short explanation of the traffic-control algorithm
7. Short explanation of traffic-state transitions
8. Instructions for demonstrating the main scenarios
9. Tests, as many as reasonable within the available time
10. A section titled "Assumptions / Questions / Requirement Issues"
11. A short section explaining major architectural decisions
12. A short section titled "AI / Tool Usage"

If AI-assisted development tools were used, briefly state which tools were used and what they were used for.

AI-assisted development is permitted. However, the candidate is responsible for understanding and being able to explain, debug, and modify the submitted solution.

19. Assumptions / Questions / Requirement Issues

Your README must include a section with this exact title:

Assumptions / Questions / Requirement Issues

Mention anything you believe is:

- Unclear
- Contradictory
- Unsafe
- Missing
- Technically problematic
- Dependent on a business decision

For each significant issue, briefly explain the decision you made.

20. Time Management

The expected effort is approximately 4-5 hours. Completing every item is not required.

Prioritize the parts you believe are most important for:

- Safety
- Correctness
- Maintainability
- Data consistency
- Recoverability
- Clear architecture

Clearly note incomplete features and explain what you would implement next with more time.

A partially complete system that preserves safety, state consistency, clear architecture, and recoverability is preferable to a visually complete application containing unsafe or tightly coupled control logic.

21. Optional Bonus Work

Only attempt bonus work if the core system is stable. Optional additions include:

- MQTT integration
- WebSocket / SSE live updates
- Docker / Docker Compose
- Authentication/authorization for manual control
- Additional automated state-machine tests
- Event replay capability
- Dead-letter/error-event handling
- Metrics
- Traffic analytics
- Configurable vehicle-priority policy
- Multiple junction instances using the same traffic engine

22. Final Note to Candidate

We are evaluating how you think as an engineer, not only how many features you complete.

Where the specification is unclear, make a reasonable decision and explain it. Where a requirement appears unsafe or contradictory, do not ignore the problem. Document it and implement the safest reasonable behavior within the available time.

You may use modern development tools, including AI-assisted coding tools. However, you are expected to understand the submitted system.

During technical review, you may be asked to:

- Explain any part of the implementation
- Diagnose unexpected behavior
- Modify an existing requirement
- Extend a small part of the system
- Correct a failure case
- Explain architectural trade-offs

The goal is not merely to generate a working demonstration. The goal is to build a small system whose behavior remains understandable, safe, and maintainable when requirements, failures, and real-world events change.